



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251172
Nama Lengkap	Ang, Joshua Alexander Hartono
Minggu ke / Materi	14 / Tipe Data Set

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pengantar

Set pada python adalah tipe data yang digunakan untuk menyimpan sekumpulan data unik atau tidak boleh sama. Set juga sering disebut sebagai himpunan karena konsepnya mirip seperti himpunan pada matematika. Isi dari sebuah set disebut anggota atau member. Salah satu ciri penting dari set adalah data yang disimpan harus bersifat immutable atau tidak bisa diubah, seperti integer, float, string, dan tuple. Karena list dan dictionary bersifat mutable, maka kedua tipe data tersebut tidak bisa dimasukkan ke dalam set. Walaupun isi data di dalam set bersifat immutable, set sendiri bersifat mutable. Artinya, isi set masih bisa ditambah atau dikurang sesuai kebutuhan. Karena set bersifat mutable, maka set tidak bisa dimasukkan ke dalam set lainnya. Hal ini dilakukan supaya data tetap aman dan tidak menimbulkan perubahan yang tidak diinginkan. Sehingga set biasanya digunakan ketika kita ingin menyimpan data tanpa duplikasi.

Dalam Python, set dapat dibuat dengan dua cara yaitu menggunakan tanda kurung kurawal {} dan menggunakan fungsi set(). Contohnya seperti {2,4,6,8} atau menggunakan set() untuk membuat set dari data tertentu. Untuk membuat set kosong sendiri, kita tidak bisa menggunakan tanda {} karena Python akan menganggapnya sebagai dictionary kosong. Oleh karena itu, set kosong harus dibuat menggunakan fungsi set(). Sedangkan jika menulis data = {}, maka hasilnya adalah dictionary kosong, bukan set.

```
# dengan menggunakan {}
bilangan_genap = {2, 4, 6, 8, 10, 12}
bilangan_ganjil = {1, 3, 5, 7, 9, 11}

# dengan menggunakan fungsi set()
pernah_ke_bulan = set('Neil Armstrong', 'Buzz Aldrin')
```

Gambar 14.1: Cara membuat set. Sumber: <https://shorturl.at/fN4UK>

```
# dengan fungsi set()
pernah_ke_mars = set() # menghasilkan set kosong

data = {} # ini akan menghasilkan dictionary kosong
```

Gambar 14.2: Cara membuat set kosong. Sumber: <https://shorturl.at/fN4UK>

Pengaksesan Set

Set pada Python tidak memiliki indeks seperti list atau tuple. Karena itu, data di dalam set tidak bisa diakses menggunakan nomor urut seperti `data[0]`. Untuk menampilkan isi set, biasanya digunakan perulangan seperti `for`. Saat isi set ditampilkan, urutannya bisa berubah-ubah dan tidak selalu sama seperti saat pertama kali dibuat. Hal ini terjadi karena pada set posisi data tidak dianggap penting.

```
nim = {'71200120', '71200195', '71200214'}
jumlah_nim = len(nim)
print(jumlah_nim)          # akan menghasilkan output 3

# tampilkan isi set satu-persatu
for n in nim:
    print(n)
```

Output yang dihasilkan dari program tersebut adalah sebagai berikut:

```
3
71200214
71200195
71200120
```

Gambar 14.3: Contoh pengaksesan set. Sumber: <https://shorturl.at/fN4UK>

Set merupakan tipe data mutable, sehingga isi datanya dapat ditambah atau dikurangi. Penambahan data dapat dilakukan menggunakan fungsi `add()`. Jika data yang ditambahkan belum ada di set, maka data tersebut akan masuk. Tetapi jika data sudah ada, maka set tidak akan menambahkan data yang sama karena set otomatis melakukan pengecekan duplikasi.

```

# definisikan sebuah set kosong
plat_nomor = set()

# tambahkan plat nomor 'AB 1890 XA'
plat_nomor.add('AB 1890 XA')

# tambahkan plat nomor 'AD 6810 MT'
plat_nomor.add('AD 6810 MT')

# jumlah anggota di dalam Set
print(len(plat_nomor))

# tambahkan plat yang sama sekali lagi
plat_nomor.add('AB 1890 XA')

# tampilkan semua plat nomor
for plat in plat_nomor:
    print(plat)

```

Gambar 14.4: Contoh penggunaan fungsi add() pada set. Sumber: <https://shorturl.at/fN4UK>

Untuk menghapus anggota set, Python menyediakan beberapa fungsi seperti remove(), discard(), pop(), dan clear(). Fungsi remove() digunakan untuk menghapus data tertentu, tetapi akan menghasilkan error jika data tidak ditemukan. Sedangkan discard() lebih aman karena tidak akan ada error walaupun data yang ingin dihapus tidak ditemukan. Fungsi pop() digunakan untuk mengambil sekaligus menghapus salah satu anggota set secara acak. Sementara itu, clear() digunakan untuk menghapus semua isi set sampai kosong.

discard()	remove()	pop()	clear()
Menghapus satu elemen yang disebutkan	Menghapus satu elemen yang disebutkan	Mengambil salah satu dan menghapusnya dari set (tidak tentu)	Menghapus seluruh elemen di dalam set
Tidak ada error	Muncul error jika elemen yang dihapus tidak ada	Error jika set kosong	Tidak ada error

Gambar 14.5: Fungsi-fungsi untuk menghapus anggota dari set. Sumber:

<https://shorturl.at/fN4UK>

```

print(bilangan_prima)

# hapus 97 (tidak ada)
bilangan_prima.discard(97)
print(bilangan_prima)

# ambil dan hapus salah satu
bilangan = bilangan_prima.pop()
print(bilangan)
print(bilangan_prima)

# kosongkan set
bilangan_prima.clear()
print(bilangan_prima)

```

Output yang dihasilkan dari program tersebut adalah:

```

{5, 7, 11, 13, 23, 29}      # awal
{7, 11, 13, 23, 29}        # setelah 5 dihapus. Perhatikan urutan berubah
{7, 11, 13, 23, 29}        # 97 tidak ada, sehingga Set tidak berubah
7                            # fungsi pop() mengeluarkan 7
{11, 13, 23, 29}           # setelah 7 keluar dari Set
set()                       # setelah isi dari Set dihapus semua

```

Gambar 14.6: Contoh penggunaan fungsi-fungsi untuk menghapus anggota set. *Sumber:*

<https://shorturl.at/fN4UK>

Pada set, nilai data tidak bisa diubah secara langsung karena set tidak memiliki indeks. Jika ingin mengganti suatu data, caranya adalah dengan menghapus data lama lalu menambahkan data baru.

```

# Buat Set dari List
ikan = set(['koi', 'koki', 'kembung', 'salmon'])
print(ikan)

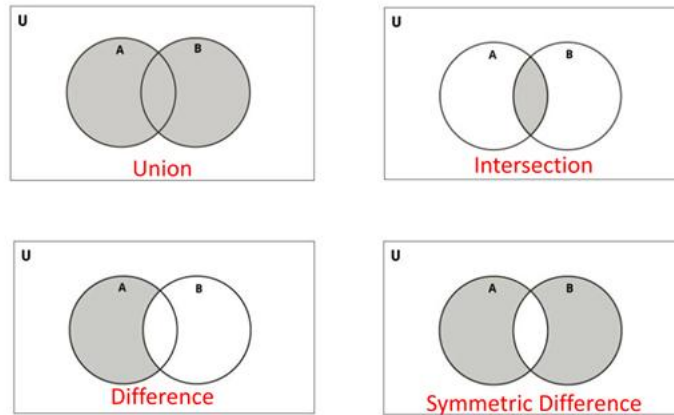
# ganti koi menjadi teri
ikan.remove('koi')
ikan.add('teri')
print(ikan)

```

Gambar 14.7: Contoh mengganti data pada set. *Sumber:* <https://shorturl.at/fN4UK>

Operasi-Operasi pada set

Pada Python, set memiliki beberapa operasi yang mirip seperti operasi himpunan pada matematika. Operasi-operasi ini digunakan untuk menggabungkan, mencari irisan, mencari selisih, dan membandingkan isi dari dua set. Operasi pada set dilakukan menggunakan simbol operator maupun fungsi bawaan Python. Hasil operasi set juga akan selalu berupa set baru.



Gambar 14.8: Ilustrasi operasi pada set. Sumber: <https://shorturl.at/fN4UK>

Operasi pertama adalah union, yaitu operasi untuk menggabungkan dua set menjadi satu. Union dapat menggunakan operator `|` atau fungsi `union()`. Saat dua set digabungkan, semua anggota dari kedua set akan masuk ke dalam set baru. Jika ada data yang sama, maka data tersebut akan muncul sekali karena set melakukan pengecekan duplikasi.

```
merek_hp = {'Samsung', 'Apple', 'Xiaomi', 'Sony'}
merek_ac = {'LG', 'Samsung', 'Panasonic', 'Daikin', 'Sony'}

# union dari merek_hp dan merek_ac
gabungan = merek_hp | merek_ac
```

Gambar 14.9: Contoh penggunaan union. Sumber: <https://shorturl.at/fN4UK>

Operasi kedua adalah intersection atau irisan. Intersection digunakan untuk mencari anggota yang sama-sama ada diantara dua set. Operasi ini dapat menggunakan operator `&` atau fungsi `intersection()`.

```
renang = {'siti', 'mail', 'ikhsan', 'upin', 'ipin'}
tenis = {'joko', 'mail', 'ipin', 'upin', 'tejo'}

# suka renang dan tenis
renang_tenis = renang & tenis
print(renang_tenis)
```

Gambar 14.10: Contoh penggunaan intersection. Sumber: <https://shorturl.at/fN4UK>

Operasi berikutnya adalah difference atau selisih. Difference digunakan untuk mencari anggota yang hanya ada di satu set dan tidak ada di set lainnya. Operasi ini menggunakan operator - atau fungsi difference()).

```
# bisa berbahasa english
english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}

# bisa berbahasa korea
korean = {'chaewon', 'yeona', 'erika', 'miko'}

# siapa yang hanya bisa bahasa korea?
only_korean = korean - english
print(only_korean)

# siapa yang hanya bisa bahasa english?
only_english = english - korean
print(only_english)
```

Gambar 14.11: Contoh penggunaan difference. Sumber: <https://shorturl.at/fN4UK>

Operasi terakhir adalah symmetric difference. Operasi ini digunakan untuk mencari anggota yang hanya ada di salah satu set, tetapi tidak termasuk anggota yang sama di kedua set. Symmetric difference menggunakan operator ^ atau fungsi symmetric_difference()).

```
english = {'desi', 'tono', 'evan', 'miko', 'takashi', 'chaewon'}
korean = {'chaewon', 'yeona', 'erika', 'miko'}

# hanya bisa bicara satu bahasa saja
one_language = english ^ korean
print(one_language)
```

Gambar 14.12: Contoh penggunaan symmetric difference. Sumber: <https://shorturl.at/fN4UK>

BAGIAN 2: LATIHAN MANDIRI (60%)

Link github : <https://github.com/angjoshua-sudo/Prak-Alpro-2026.git>

SOAL 1

Source code:

```
1  n = int(input("Masukkan jumlah kategori: "))
2
3  data_aplikasi = {}
4  for i in range(n):
5      nama_kategori = input("Masukkan nama kategori: ")
6      print('Masukkan 5 nama aplikasi di kategori', nama_kategori)
7
8      aplikasi = []
9      for j in range(5):
10         nama_aplikasi = input("Nama aplikasi: ")
11         aplikasi.append(nama_aplikasi)
12
13     data_aplikasi[nama_kategori] = aplikasi
14
15     daftar_aplikasi_list = []
16
17     for aplikasi in data_aplikasi.values():
18         daftar_aplikasi_list.append(set(aplikasi))
19
20     hasil = daftar_aplikasi_list[0]
21     for i in range(1, len(daftar_aplikasi_list)):
22         hasil = hasil.intersection(daftar_aplikasi_list[i])
23
24     tabel = {}
25     for i in daftar_aplikasi_list:
26         for j in i:
27             tabel[j] = tabel.get(j, 0) + 1
28
29     unik = []
30     sama2 = []
31     for key, value in tabel.items():
32         if value == 2:
33             sama2.append(key)
34         elif value == 1:
35             unik.append(key)
36
37     print(f"Aplikasi yang muncul di semua kategori adalah: \n{hasil}")
38     print(f"Aplikasi yang muncul di satu kategori adalah: \n{unik}")
39     print(f"Aplikasi yang muncul di dua kategori adalah: \n{sama2}")
```

Penjelasan:

Pada bagian awal program, pengguna diminta memasukkan jumlah kategori yang ingin diproses menggunakan variabel **n**. Setelah itu dibuat sebuah dictionary kosong bernama **data_aplikasi** yang nantinya akan menyimpan setiap kategori beserta daftar aplikasinya. Di dalam perulangan sebanyak **n** kali, program meminta input nama kategori dari user, lalu menampilkan instruksi untuk memasukkan 5 nama aplikasi pada kategori tersebut. Setiap aplikasi yang dimasukkan akan

disimpan ke dalam sebuah list, kemudian list tersebut akan disimpan ke dalam dictionary dengan key berupa **nama_kategori**.

Pada bagian selanjutnya, program membuat list baru bernama **daftar_nilai_list** yang berfungsi untuk menyimpan semua daftar aplikasi dalam bentuk set. Set ini digunakan agar data aplikasi tidak memiliki duplikasi dalam satu kategori dan memudahkan proses operasi himpunan. Setelah itu, program melakukan perulangan terhadap seluruh value dari dictionary **data_aplikasi** dan mengubahnya menjadi set sebelum dimasukkan ke dalam list tersebut. Kemudian variabel **hasil** diisi dengan set pertama dan selanjutnya dilakukan operasi intersection untuk mencari aplikasi yang muncul di semua kategori.

Pada tahap terakhir, program membuat dictionary bernama **table** untuk menghitung frekuensi kemunculan setiap aplikasi di semua kategori. Setiap aplikasi yang ditemukan akan ditambahkan nilainya, sehingga dapat diketahui berapa kali aplikasi tersebut muncul. Setelah itu, dibuat dua list yaitu **unik** dan **sama2** untuk mengelompokkan aplikasi berdasarkan jumlah kemunculannya. Aplikasi yang muncul dua kali dimasukkan ke **sama2**, sedangkan aplikasi yang hanya muncul sekali dimasukkan ke **unik**. Terakhir, program akan menampilkan hasil berupa aplikasi yang muncul di semua kategori, di satu kategori saja, dan di dua kategori menggunakan format **fstring**.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu13> & C:\Users\Acer\AppData\Local\Programs\Python\Python313\python.exe c:/Users/Acer/OneDrive/Dokumen/belajar-github/minggu14/soal1.py
Masukkan jumlah kategori: 3
Masukkan nama kategori: Finance
Masukkan 5 nama aplikasi di kategori Finance
Nama aplikasi: IPOT
Nama aplikasi: Mirae
Nama aplikasi: Calculator
Nama aplikasi: Notes
Nama aplikasi: Zoom
Masukkan nama kategori: Utilities
Masukkan 5 nama aplikasi di kategori Utilities
Nama aplikasi: Calculator
Nama aplikasi: Camera
Nama aplikasi: Notes
Nama aplikasi: Weather
Nama aplikasi: Maps
Masukkan nama kategori: Education
Masukkan 5 nama aplikasi di kategori Education
Nama aplikasi: Zoom
Nama aplikasi: Classroom
Nama aplikasi: Calculator
Nama aplikasi: Docs
Nama aplikasi: Meet
Aplikasi yang muncul di semua kategori adalah:
{'Calculator'}
Aplikasi yang muncul di satu kategori adalah:
['Mirae', 'IPOT', 'Camera', 'Weather', 'Maps', 'Docs', 'Classroom', 'Meet']
Aplikasi yang muncul di dua kategori adalah:
['Notes', 'Zoom']
```

SOAL 2

Source code:

```
1  print("Pilihan Konversi:")
2  print("1. Konversi List menjadi Set")
3  print("2. Konversi Set menjadi List")
4  print("3. Konversi Tuple menjadi Set")
5  print("4. Konversi Set menjadi Tuple")
6  angka = int(input("Masukkan angka untuk memilih jenis konversi: "))
7
8  masuk = eval(input("Masukkan data: "))
9
10 if angka == 1:
11     print(f"Type data: List")
12     print(masuk)
13     hasil = set(masuk)
14     print("Type data: Set")
15     print(hasil)
16 elif angka == 2:
17     print(f"Type data: Set")
18     print(masuk)
19     hasil = list(masuk)
20     print("Type data: List")
21     print(hasil)
22 elif angka == 3:
23     print(f"Type data: Tuple")
24     print(masuk)
25     hasil = set(masuk)
26     print("Type data: Set")
27     print(hasil)
28 elif angka == 4:
29     print(f"Type data: Set")
30     print(masuk)
31     hasil = tuple(masuk)
32     print("Type data: Tuple")
33     print(hasil)
34 else:
35     print("Pilihan tidak valid")
```

Penjelasan:

Pada bagian awal program, pada baris pertama sampai baris kelima digunakan untuk menampilkan menu pilihan konversi tipe data kepada user. Program akan mencetak beberapa pilihan seperti konversi List menjadi Set, Set menjadi List, Tuple menjadi Set, dan Set menjadi Tuple menggunakan fungsi **print()**. Setelah semua pilihan ditampilkan, program akan meminta user memasukkan angka pilihan menggunakan fungsi **input()**. Kemudian nilai tersebut disimpan ke variabel **angka**.

Selanjutnya, program pada baris ke delapan akan meminta user memasukkan data yang ingin dikonversi dan hasil input akan disimpan ke variabel **masuk**. Input menggunakan fungsi **eval()** agar data yang masuk dapat langsung dibaca sebagai tipe data oleh Python. Setelah itu, program akan mulai menggunakan percabangan **if**, **elif**, dan **else** untuk memeriksa pilihan angka yang dimasukkan sebelumnya. Pada bagian kondisi **if angka == 1**, program akan mengubah data list menjadi set menggunakan fungsi **set(masuk)** dan hasilnya disimpan ke variabel **hasil**. Setelah konversi selesai, program menampilkan tulisan "**Tipe data: Set**" lalu mencetak isi hasil konversi tersebut. Penggunaan set membuat data yang memiliki duplikat otomatis dihilangkan. Pada kondisi berikutnya yaitu **elif angka == 2**, program akan melakukan proses kebalikan yaitu mengubah set menjadi list menggunakan fungsi **list(masuk)**. Hasil konversi kemudian dicetak kembali agar user dapat melihat perubahan tipe datanya. Pada kondisi **elif angka == 3**, program akan mengubah tipe data tuple menjadi set menggunakan fungsi **set()**. Program juga menampilkan tipe data awal yaitu tuple sebelum hasil konversi dicetak ke layar. Kemudian pada kondisi **elif angka == 4**, program mengubah set menjadi tuple menggunakan fungsi **tuple(masuk)**. Pada bagian akhir terdapat kondisi **else** yang digunakan untuk menangani user memasukkan angka selain 1 sampai 4 sehingga program akan mencetak pesan "**Pilihan tidak valid**".

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu14> py soal2.py
Pilihan Konversi:
1. Konversi List menjadi Set
2. Konversi Set menjadi List
3. Konversi Tuple menjadi Set
4. Konversi Set menjadi Tuple
Masukkan angka untuk memilih jenis konversi: 1
Masukkan data: [1, 2, 3, 3, 4]
Tipe data: List
[1, 2, 3, 3, 4]
Tipe data: Set
{1, 2, 3, 4}
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu14> py soal2.py
Pilihan Konversi:
1. Konversi List menjadi Set
2. Konversi Set menjadi List
3. Konversi Tuple menjadi Set
4. Konversi Set menjadi Tuple
Masukkan angka untuk memilih jenis konversi: 2
Masukkan data: {10, 20, 30}
Tipe data: Set
{10, 20, 30}
Tipe data: List
[10, 20, 30]
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu14> py soal2.py
Pilihan Konversi:
1. Konversi List menjadi Set
2. Konversi Set menjadi List
3. Konversi Tuple menjadi Set
4. Konversi Set menjadi Tuple
Masukkan angka untuk memilih jenis konversi: 3
Masukkan data: (7, 8, 9, 9)
Tipe data: Tuple
(7, 8, 9, 9)
Tipe data: Set
{8, 9, 7}
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu14> py soal2.py
Pilihan Konversi:
1. Konversi List menjadi Set
2. Konversi Set menjadi List
3. Konversi Tuple menjadi Set
4. Konversi Set menjadi Tuple
Masukkan angka untuk memilih jenis konversi: 4
Masukkan data: {100, 200, 300}
Tipe data: Set
{200, 100, 300}
Tipe data: Tuple
(200, 100, 300)
```

SOAL 3

Source code:

```
1  import string
2  file1 = input('Masukkan nama file pertama: ')
3  file2 = input('Masukkan nama file kedua: ')
4  try:
5      handle = open(file1)
6      handles = open(file2)
7  except:
8      print("File tidak ditemukan atau tidak bisa dibaca")
9      exit()
10
11  kata1 = []
12  for line in handle:
13      line = line.translate(str.maketrans('', '', string.punctuation))
14      line = line.lower()
15      pisah = line.split()
16      for kata in pisah:
17          kata1.append(kata)
18
19  kata2 = []
20  for line in handles:
21      line = line.translate(str.maketrans('', '', string.punctuation))
22      line = line.lower()
23      pisah = line.split()
24      for kata in pisah:
25          kata2.append(kata)
26
27  kata1 = set(kata1)
28  kata2 = set(kata2)
29
30  gabung = kata1 & kata2
31  print(gabung)
32  handle.close()
33  handles.close()
```

Penjelasan:

Pada bagian awal program, baris pertama menggunakan **import string** untuk mengambil library string yang akan dipakai dalam penghapusan tanda baca. Setelah itu, pada baris kedua dan ketiga, program meminta user memasukkan nama file pertama dan file kedua menggunakan **input()**. Nama file tersebut kemudian disimpan ke dalam variabel **file1** dan **file2**. Selanjutnya program menggunakan blok try untuk mencoba membuka kedua file menggunakan fungsi **open()**. Jika file tidak ditemukan atau tidak dapat dibaca, maka bagian **except** akan dijalankan sehingga program akan menghentikan program menggunakan **exit()**.

Pada bagian berikutnya, program akan membuat list kosong bernama **kata1** untuk menyimpan semua kata dari file pertama. Program membaca isi file menggunakan perulangan **for line in handle** sehingga file diproses baris demi baris. Setiap baris kemudian dibersihkan dari tanda baca menggunakan **translate()** dan **str.maketrans()** dengan bantuan **string.punctuation**. Setelah itu semua huruf diubah menjadi huruf kecil menggunakan **lower()** agar format kata menjadi seragam. Baris yang sudah dibersihkan lalu dipisahkan menjadi beberapa kata menggunakan **split()** dan setiap kata dimasukkan ke dalam list **kata1** menggunakan **append()**. Proses yang sama juga dilakukan pada file kedua dengan menggunakan list **kata2** sehingga kedua file memiliki format data yang sama sebelum dibandingkan.

Pada bagian akhir program, list **kata1** dan **kata2** diubah menjadi tipe data set menggunakan fungsi **set()**. Penggunaan set bertujuan agar kata yang sama hanya disimpan satu kali dan mempermudah proses pencarian kata yang muncul di kedua file. Setelah itu program menggunakan operator **&** untuk mencari irisan dari kedua set dan hasilnya disimpan ke variabel **gabung**. Selanjutnya, program mencetak isi dari variabel **gabung** menggunakan **print()**. Terakhir, program menutup kedua file menggunakan method **close()**.

Output:

```
PS C:\Users\Acer\OneDrive\Dokumen\belajar-github\minggu14> py soal3.py
Masukkan nama file pertama: soal3a.txt
Masukkan nama file kedua: soal3b.txt
{'karel', 'lelah', 'kapan', 'mau', 'saya', 'pak'}
```